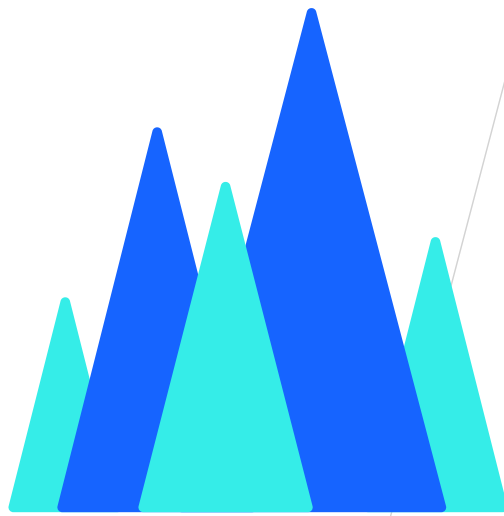




ByteDance veRoCE Transport Protocol

Version: 2.0.4

ByteDance
Dec 18, 2025

Table 1: Revision History

Version	Date	Author	Description
2.0.4	December 18, 2025	ByteDance	Initial public version

Legal Disclaimer

Copyright © 2025 ByteDance Ltd. and/or its affiliates.

This document is a specification ("Specification") of ByteDance veRoCE Transport Protocol.

The Specification is made available under the terms of the Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License (CC BY-NC-ND 4.0). This license enables you to copy and distribute the material in any medium or format in unadapted form only, for noncommercial purposes only, and only so long as attribution is given to ByteDance veRoCE Transport Protocol. A copy of the license is available at <https://creativecommons.org/licenses/by-nc-nd/4.0/>. This Specification is provided "AS IS" without any warranties of any kind, either express or implied, including but not limited to warranties of non-infringement, merchantability, accuracy, completeness, or fitness for a particular purpose. In no event shall ByteDance Ltd. and/or any of its affiliates be liable for any damages, losses, or liabilities of any kind, whether direct, indirect, special, exemplary, punitive, or consequential, including, without limitation, lost profits, arising from or in connection with this specification, even if advised of the possibility of such damages.

ByteDance veRoCE Transport Protocol reserves the right to adopt any changes or alterations to this Specification as it deems necessary or appropriate.

For more information about ByteDance veRoCE Transport Protocol, please contact Huichen Dai (daihuichen@bytedance.com).

Contents

1	Overview	1
2	Terminology	3
3	Transport Headers	6
3.1	General Packet Format	6
3.2	CRC Generation and Checking Rules	6
3.3	Base Transport Header (BTH)	7
3.4	MSN Extended Transport Header (MSNETH)	7
3.5	Packet Offset Extended Transport Header (POETH)	8
3.6	RQ Extended Transport Header (RQETH)	8
3.7	ACK Extended Transport Header (AETH)	9
3.8	SACK Extended Transport Header (SACKETH)	9
3.9	RTT probing Request Extended Transport Header (RTTReqETH)	10
3.10	RTT probing Response Extended Transport Header (RTTRspETH)	11
4	Transport Semantics	12
5	Reliable Connection Service	13
5.1	PSN	13
5.2	MSN	14
5.3	Acknowledgement Protocol	16
5.3.1	ACK/SACK Protocol	16
5.3.2	NAK Protocol	17
5.3.3	ACK Example	18
5.3.4	SACK Example	18
5.3.5	Packet Drop Detection	19
5.3.6	Fast Selective Retransmission	19
5.4	Send/Write Operations	21
5.4.1	Requester: Generating Request Packets	23
5.4.2	Responder: Receiving Request Packets	24
5.4.3	Responder: Generating Acknowledgement Packets	24
5.4.4	Responder: Message Completion	24
5.4.5	Requester: Message Completion	25
5.5	Read Operation	26
5.5.1	Requestor: Generating Read Request Packet	27
5.5.2	Responder: Receiving Read Request Packet	27
5.5.3	Responder: Generating Read Response Packet	28
5.5.4	Responder: Supporting Read Response DDP	28
5.5.5	Requester: Receiving Read Response Packets	28
5.5.6	Requester: Generating Acknowledgement Packets & Message Completion	28

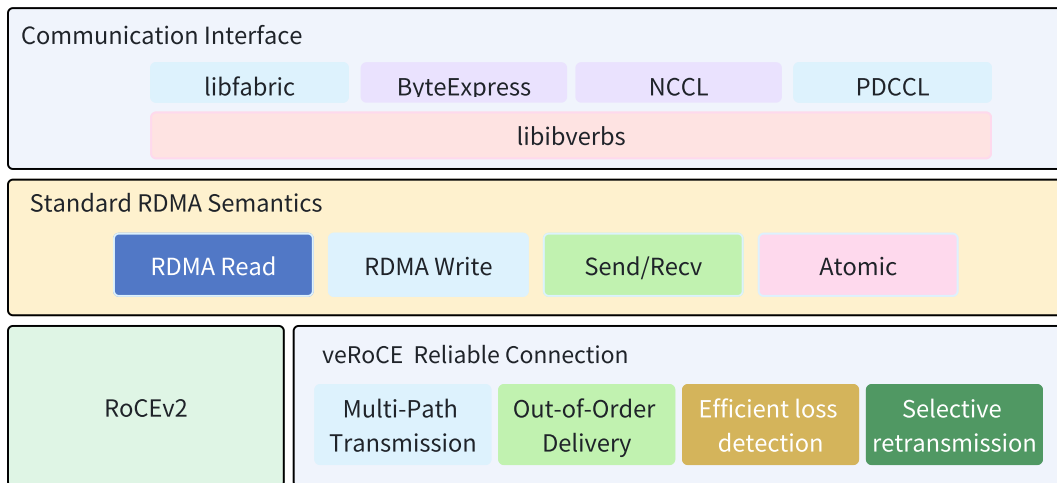
5.5.7	Responder: Receiving Acknowledgement Packets & Releasing buffered Read Request	28
5.6	Atomic Operation	29
5.7	Operation Ordering	30
5.8	Shared Received Queue	30
5.9	Packet Trimming	31
6	Error Handling	32
6.1	Removed Error Behaviors	32
6.1.1	Requestor-Side Removed Errors	32
6.1.2	Responder-Side Removed Errors	32
6.2	Added Error Behaviours	32
6.2.1	Requestor-Side Added Behaviours	32
6.2.2	Responder-Side Added Behaviors	33
7	Congestion Control	34
7.1	Congestion Notification	34
7.2	RTT Probing	34
7.3	Congestion Control Modes	35
7.4	Slow Path Detection	36
8	Feature Profiles	37
9	Communication Management	38
9.1	Vendor ID Encoding	38
9.2	Negotiation Process	38
9.3	QPC Resource allocation	38
10	Programmer's Guide to veRoCE	39
10.1	Operation Ordering Considerations	39
10.2	Enhanced Application Programming Interfaces	39

1 Overview

In the artificial intelligence (AI) era, RDMA (Remote Direct Memory Access) is widely used for high-speed, low-latency data movement among GPUs or AI accelerators. InfiniBand (IB) RDMA enables efficient communication over InfiniBand networks, while RoCEv2 (RDMA over Converged Ethernet) extends RDMA to Ethernet-based data centers. However, both IB and RoCEv2 require a lossless network to operate effectively, making RDMA transport protocols inefficient at handling packet drops and out-of-order packet arrivals. Additionally, lossless network fabrics are prone to congestion spreading and deadlocks, which severely limit network scale. As AI networks will expand to support hundreds of thousands, or even millions, of GPUs or AI accelerators, these limitations become more pronounced. At such a scale, packet drops and out-of-order arrivals caused by multi-path transmission and lossy networks are unavoidable 'facts' that must be addressed.

This specification defines the veRoCE transport protocol, a solution that maintains backward compatibility with RoCEv2 while introducing key extensions to enhance RDMA's resilience to packet drops and out-of-order arrivals across all transport semantics. By addressing these limitations, veRoCE eliminates RDMA's reliance on a lossless network. The success of RDMA largely comes from its well-designed message semantics (e.g., RDMA READ/WRITE), packet formats, and user interfaces (e.g., ibverbs). To preserve these strengths, veRoCE aims to minimize modifications to the existing RDMA protocol while introducing only necessary enhancements.

Specifically, veRoCE features multi-path transmission, out-of-order delivery, efficient packet loss detection, hardware-friendly selective retransmission, and flexible congestion control for all transport semantics. Multi-path transmission in veRoCE can be achieved via changing packet entropy by the source endpoint (and relying on switch ECMP), or spraying packets on the switch. Out-of-order data delivery is natively supported at the responder endpoint, as the data packets contain extended headers to support DDP (Direct Data Placement), even for Send/Recv operations. veRoCE implements selective acknowledgments (SACKs) to notify the sender of out-of-order packet arrivals. This mechanism enables rapid detection of packet loss and triggers a fast selective retransmission algorithm that efficiently recovers lost packets while minimizing unnecessary retransmissions. Unlike IB/RoCE, where Read response packets consume PSNs from the requester, veRoCE assigns a separate PSN space for Read responses at the responder side. Upon receiving these responses, the requester sends corresponding ACKs or SACKs. This design unifies the packet loss detection and fast retransmission mechanisms for both Send/Write packets and Read responses. veRoCE provides a FCC (Flexible Congestion Control) framework to allow incorporation of customized congestion control algorithms. It supports path-wise or connection-wise, window-based or rate-based congestion control modes. In rate-based congestion control, it decouples congestion control from transport reliability, which means the congestion signals are conveyed in separate packets rather than the commonly used acknowledgement packets. Thus, the coalescing of acknowledgement packets on an endpoint will not be interrupted by the requirement to transmit a congestion signal. veRoCE also supports unreliable transmission, primarily for connection management packets. The communication interface that veRoCE natively supports is libibverbs, upon which other high-level APIs can be built.



Additional Conventions:

- **Byte Ordering:** This specification uses Big Endian byte ordering. For fields larger than one byte, the most significant byte is transmitted first.
- **Numerical Notation:** Unless otherwise stated, unqualified numerical values are decimal. Qualifiers follow these rules:
 - **Hexadecimal values:** Prefix with `0x...` (e.g., `0xFF`); ranges are written as `0xA-0xF`.
 - **Binary values:** Prefix with `b'...` (e.g., `b'010`); ranges are written as `b'010-b'111`.

2 Terminology

Term	Definition
QP (Queue Pair)	A QP consists of a Send Queue (SQ) and a Receive Queue (RQ). SQ and RQ are always created as a pair and remain that way throughout their lifetime. A QP is identified by its QP Number.
Send Queue (SQ)	A Work Queue, that contains descriptors (or WQEs, see below) for outbound messages, including RDMA Read Request, RDMA Write messages and Send messages, in the order as the descriptors are posted from applications.
Receive Queue (RQ)	A Work Queue, that contains descriptors (or WQEs, see below) that specify where to place inbound data.
Completion Queue (CQ)	A Work Queue, that contains Completion Queue Elements that represent message completion. A CQ can be multiplexed across multiple QPs.
Work Queue	One of SQ, RQ, CQ.
Work Queue Element (WQE)	An item in the Work Queue, usually a descriptor for requests, data, or message completion.
RQE	Receive Queue Element, same as Receive Queue WQE.
Requester	The QP that initiates the RDMA Read, RDMA Write and Send operations. It sends out the request packets of RDMA Read, RDMA Write and Send operations. It may more broadly represent the thread, process, NIC or the host that this QP belongs to.
Responder	The QP that accepts request packets of RDMA Read, RDMA Write and Send operations, processes the messages, and returns responses. It may more broadly represent the thread, process, NIC or the host that this QP belongs to. Requester and responder are usually used as a pair.
Request (Packet)	Packets that require the receiving endpoint to process them (e.g., do DMAs) and generate response packets, generated by requesters and received by responders. Request packets include Read Request, Write Request, Send Request.
Response (Packet)	Packets generated by a responder in response to the reception of request packets. Response packets include acknowledgement packets (see below) and Read Responses. Request and Response are usually used as a pair.
Semantic Packet	Packets that carry semantic meanings, including RDMA Read Request, RDMA Read Response, RDMA Atomic Request, RDMA AtomicAck, RDMA Write Request, and Send Request.
Acknowledgement Packet (ack_pkt)	Control packets used by a receiver (see below) to acknowledge (positively or negatively) the semantic packets, including ACK, NAK, and SACK packets. Semantic Packet and Acknowledgement Packet are usually as a pair.

Term	Definition
Sender	Refer to the QP that sends semantic packets. It will receive acknowledgement packets to ascertain the delivery of semantic packets. Therefore, it is legal to use the phrase "A sender transmits a Write request packet and then receives an ACK."
Receiver	Refer to the QP that receives semantic packets. It will transmit acknowledgement packets to inform the sender about the arrival of semantic packets. Therefore, it is legal to use the phrase "A receiver receives a Write request packet and then transmits an ACK."
ACK	Positive acknowledgement packet, used to ascertain the arrival of one or more in-order semantic packets (except Read Responses), sent by receiver to sender.
SACK	Selective Acknowledgement packet, used to selectively ascertain the arrival of both in-order and out-of-order semantic packets (except Read Responses).
NAK	Negative Acknowledgement packet, used to explicitly inform the sender that one semantic packet (except Read Responses) has been dropped, possibly dropped by a switch or a receiver (due to insufficient resource).
ACK_Rsp	Positive acknowledgement packet, used to ascertain the arrival of one or more Read Response or AtomicAck packets, sent by receiver to sender.
SACK_Rsp	Selective Acknowledgement packet for Read Responses and AtomicAck packets.
NAK_Rsp	Negative Acknowledgement packet, used to explicitly inform the sender that one Read Response or Atomic packet has been dropped, possibly dropped by a switch or a receiver (due to insufficient resource).
PSN (Packet Sequence Number)	A monotonically increasing sequence number assigned to semantic packets by senders.
aPSN (Acknowledgement PSN)	The most recent sequence number such that all packets with PSNs less than or equal to this value have been received.
ePSN (Expected PSN)	The PSN of the next in-order semantic packet that a receiver expects to receive, i.e., $aPSN + 1$. This indicates that the packet of ePSN must not have been received, and PSNs larger than ePSN may have been received.
MSN (Message Sequence Number)	A monotonically increasing sequence number assigned to messages initiated from both the requestor and the responder. Request messages from the requestor and Response messages from the responder are in the different MSN spaces.
aMSN (Acknowledgement Packet MSN)	The most recent sequence number such that all messages with MSNs less than or equal to this value have been received.

Term	Definition
RQMSN (Receive Queue Aligned Message Sequence Number)	A monotonically increasing sequence number assigned to messages that consume a receive WQE. The assigner is requester (for Send, Send-with-ImmDt and Write-with-ImmDt).

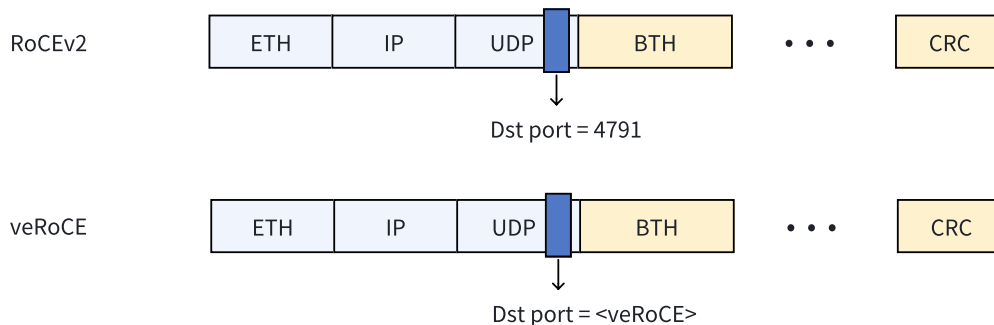
3 Transport Headers

This chapter details the packet format and transport headers of veRoCE, with a focus on differences from RoCEv2. All headers are defined with clear field descriptions and usage rules to ensure implementability.

3.1 General Packet Format

veRoCE packets use a dedicated UDP destination port number allocated by the Internet Assigned Numbers Authority (IANA) (presumably 4794), while RoCEv2 packets use UDP destination port 4791. A 32-bit Cyclic Redundancy Check (CRC) covers all invariant fields in the IP Header, UDP Header, Base Transport Header, Extended Transport Headers, and packet payload—this ensures data integrity during transmission.

The packet structure comparison between RoCEv2 and veRoCE is as follows:



3.2 CRC Generation and Checking Rules

The rules for generation/checking of the CRC subject to:

1. The ICRC calculation starts with 64 bits of '1'.
2. The ICRC calculation continues with the entire IP datagram starting with the first byte of the IP header up until and including the last veRoCE Payload byte right before the CRC field itself.
3. The variant fields in the IP header are replaced with '1s for the purpose of the CRC calculation/check so that changes to these fields along the way don't affect the calculated CRC value. For veRoCE over IPv4, the fields replaced with '1s for the purpose of ICRC calculation are:
 - Time to Live
 - Header Checksum
 - Type of Service (DSCP and ECN)

For veRoCE over IPv6, the fields replaced with '1s for the purpose of ICRC calculation are:

- Traffic Class (DSCP and ECN)
- Flow Label
- Hop Limit

- The UDP Checksum field is replaced with '1s for the purpose of the CRC calculation/check.

In the following, only the modified or added transport headers in veRoCE are described, compared with RoCEv2.

3.3 Base Transport Header (BTH)

The BTH is included in all veRoCE packets and retains the same format as defined in the IB specification. Most Opcodes remain unchanged from IB, except for SACK, ACK_Rsp, and SACK_Rsp. However, the assignment of PSNs to RDMA Read Requests and Responses differs.

31-24				23-16				15-8				7-0			
Opcode				SE	M	Pad	TVer		Partition Key						
FECN	BECN	Rsvd		Destination QP											
A	RE	Rsvd		PSN											

Field	Width	Values	Description
Opcode	8 bit	Opcode[4:0] defines operation types For Opcode[7:5]=b'000: - b'11000: SACK - b'11001: ACK_Rsp - b'11110: SACK_Rsp - b'11111: Reserved For Opcode[7:5]=b'100: - b'00010: RTT Req - b'00011: RTT Rsp - b'00100: Slow path Others are the same as RoCEv2	veRoCE introduces new opcodes of SACK, ACK_Rsp, SACK_Rsp, RTT Request/Response, and Slow path. veRoCE also defines a new packet type of NAK_Rsp for Read Response packets, but it shares the same opcode with ACK_Rsp. All other opcode values remain unchanged.
PSN	24 bit	ACK的PSN不独立递增了？ (好像有别于IB)	The PSN associated with a packet. - For semantic packets: a monotonically increasing sequence number assigned to packets by the sender. - For ACK/SACK/ACK_Rsp/SACK_Rsp: populated with the PSN of the most recent request packet being contiguously acknowledged. - For NAK/NAK_Rsp: PSN of the request packet that causes the error.
Retrans (Re)	1 bit	0 or 1	Retransmission flag. 1 indicates a retransmitted packet, 0 indicates an original packet.

3.4 MSN Extended Transport Header (MSNETH)

Two extended headers, MSNETH and POETH, are introduced to facilitate completion processing.

31-24	23-16	15-8	7-0
Rsvd	MSN		

Fields	Width	Description
MSN	24 bit	For Send, Write, and Read request messages: The MSN assigned by the requestor. For Read response and AtomicAck messages: The MSN assigned by the responder.
Rsvd	8 bit	Reserved field, transmitted as 0 and ignored on the receiver.

3.5 Packet Offset Extended Transport Header (POETH)

POETH is used for DDP of Send and RDMA Read operations, and accelerating the retransmission process for RDMA Write operation.

31-24	23-16	15-8	7-0
Rsvd	In-message Packet Order		

搭配RETH使用
RETH是起始地址，
POETH是偏移
如Mid/Last必带

Fields	Width	Values	Description
In-Message Packet Order (PO)	24 bit	0 - 0x7FFFFF: Packet order 0x800000 - 0xFFFFFE: Reserved 0xFFFFFFFF: Not supported	For semantic packets, this field is populated with the packet order/offset inside the message it belongs to, starting from 0. For Send Operation, it is used to compute the address where the payload should be placed on the responder side, offsetting from the base address specified by the corresponding RQE. For ack_pkt, the PO field is populated by the receiver with the PO value corresponding to aPSN + 1. This field helps quickly locate the data to be transmitted for a WQE. Specifically, SACK.PO is used for fast selective retransmission, while ACK.PO is used for RTO retransmission. Especially, for ack_pkts, the value 0xFFFFFFFF means packet offset information is not available, and the sender should locate the data to be retransmitted solely on PSN. For semantic packets, this field must always be populated with valid values.
Rsvd	8 bit	0x00	Reserved field. Always transmitted as 0 and ignored by the receiver.

3.6 RQ Extended Transport Header (RQETH)

The RQETH is used to associate a Send or Write-with-ImmDt with a specific RQE at the responder.

RQMSN用于找到对端的WQE（按理说，开逐包时可能乱序，此时需要提前找到）
类似于UB的TaskID ETAH字段？

31-24	23-16	15-8	7-0
Rsvd	RQMSN		

Fields	Width	Description
RQMSN	24 bit	The RQMSN associated with the message, starting from 0.
Rsvd	8 bit	Reserved field, transmitted as 0 and ignored on the receiver.

3.7 ACK Extended Transport Header (AETH)

The definitions of MSN and Syndrome are **modified** due to the introduction of SACK and enhanced Read mechanism.

31-24	23-16	15-8	7-0
Syndrome	MSN		

IB中的AETH头只有PSN字段值，用于确认请求端的PSN
veRoCE里PSN确认被放到了BTH.PSN，AETH则是用来确认MSN？

Field	Width	Values	Description
MSN	24 bit		- For ack_pkt (Packet Drop NAK excluded): populated with aMSN at the receiver. - For Packet Drop NAK: populated with the MSNETH.MSN in the trimmed packet. - For Read Response and AtomicAck: populated with the MSN of the associated request. It is used by the requester to correlate a response packet to an outstanding request.
Syndrome	8 bit	- b'01100000: PSN-Error NAK in RoCEv2 is repurposed to Packet Drop NAK in veRoCE - Others keep the same meaning as RoCEv2	- For ACK/ACK_Rsp/NAK/NAK_Rsp: the sequence error NAK in RoCEv2 is repurposed as Packet Drop NAK. - For Read Response and AtomicAck: the syndrome is unused. - For SACK/SACK_Rsp: only the flow control credit is valid, the rest of the syndromes are unused.

3.8 SACK Extended Transport Header (SACKETH)

The SACKETH is introduced to selectively acknowledge out-of-order packet arrivals.

31-24	23-16	15-8	7-0
-------	-------	------	-----

Bitmap Valid Len	Bitmap Starting PSN
PSN bitmap[127:96]	
PSN bitmap[95:64]	
PSN bitmap[63:32]	

Field	Width	Description
Bitmap Starting PSN	24 bit	The PSN corresponds to the first bit in the bitmap. By default, this is the PSN of the most recently contiguously acknowledged request packet.
Bitmap Valid Length	8 bit	The valid length of the bitmap in bits, starting from the LSB in the PSN bitmap field.
PSN bitmap	128 bit	A bitmap where 1s indicate received packets. The PSN range covered by the bitmap is [SACKETH.Bitmap_starting_psn, SACKETH.Bitmap_starting_psn + SACKETH.Bitmap_Valid_length - 1]. The 1s shall only be used for retransmission but not for acknowledging packets.

3.9 RTT probing Request Extended Transport Header (RTTReqETH)

RTTReqETH is used for Round-Trip Time (RTT) probing — a key capability for congestion control. It carries timestamps and context information to measure network latency accurately.

Field	Width	Description
CC context ID	32bit	A unique identifier for a CC context that issues the RTT probing request, which can be defined with varying levels of granularity, such as per path, per QP, or per destination IP address.
TX timestamp1	32bit	The timestamp when the RTT probing request packet is transmitted by the sender (in nanoseconds since a fixed epoch).
RX timestamp1	32bit	The timestamp when the RTT probing request packet is received by the receiver (in nanoseconds since a fixed epoch). Must be set to 0 when sending the RTT probing request.
TX timestamp2	32bit	The timestamp when the RTT probing response packet is transmitted by the receiver (in nanoseconds since a fixed epoch). Must be set to 0 when sending the RTT probing request.
RX timestamp2	32bit	The timestamp when the RTT probing response packet is received by the sender (in nanoseconds since a fixed epoch). Must be set to 0 when sending the RTT probing request.

3.10 RTT probing Response Extended Transport Header (RTTRspETH)

RTTRspETH is the response to RTTReqETH. It echoes the CC context ID from the request and fills in receiver-side timestamps, enabling the sender to calculate RTT. RTTRspETH and RTTReqETH share the same field format; they are separated into two headers only to distinguish requests from responses.

Field	Width	Description
CC context ID	32bit	Echos the CC context ID of the corresponding RTT probing request packet (ensures the response is linked to the correct request).
TX timestamp1	32bit	The TX Timestamp1 value from the received RTTReqETH (echoed to the sender for latency calculation).
RX timestamp1	32bit	The timestamp when the RTT probing request packet is received by the receiver (in nanoseconds since a fixed epoch).
TX timestamp2	32bit	The timestamp when the RTT probing response packet is transmitted by the receiver (in nanoseconds since a fixed epoch).
RX timestamp2	32bit	The timestamp when the RTT probing response packet is received by the sender (in nanoseconds since a fixed epoch). Must be set to 0 when sending the RTT probing response.

4 Transport Semantics

veRoCE supports a comprehensive set of transport semantics to meet the requirements of different RDMA operations (e.g., Read, Write, Atomic). The table below details the supported operations, their corresponding packet opcodes, and packet formats for Reliable Connection (RC) type connections, which is veRoCE's primary connection type for lossy networks.

Connection Type	Operation	Packet Opcode	Packet Format
RC	RDMA READ	Read Request	BTH, MSNETH, RETH
		Read Response First/Only	BTH, MSNETH, AETH, Payload
		Read Response Middle/Last	BTH, MSNETH, AETH, POETH, Payload
	RDMA WRITE	Write First/Only	BTH, MSNETH, RETH, Payload
		Write Middle/Last	BTH, MSNETH, RETH, POETH, Payload
		Write-with-ImmDt Last	BTH, MSNETH, RETH, RQETH, POETH, ImmDt, Payload
		Write-with-ImmDt Only	BTH, MSNETH, RETH, RQETH, ImmDt, Payload
	SEND	Send First/Only	BTH, MSNETH, RQETH, Payload
		Send Middle/Last	BTH, MSNETH, RQETH, POETH, Payload
		Send-with-ImmDt Last	BTH, MSNETH, RQETH, POETH, ImmDt, Payload
		Send-with-ImmDt Only	BTH, MSNETH, RQETH, ImmDt, Payload
		Send Only with Invalidate	BTH, MSNETH, RQETH, IETH, Payload
		Send Last with Invalidate	BTH, MSNETH, RQETH, POETH, IETH, Payload
	ATOMIC	CmpSwap/ FetchAdd	BTH, MSNETH, AtomicETH
	ACK/NAK	ACK/ ACK_Rsp	BTH, AETH, POETH
	ATOMIC ACK	ATOMIC ACK	BTH, MSNETH, AETH, AtomicAckETH
	SACK	SACK/SACK_Rsp	BTH, AETH, POETH, SACKETH
Congestion Signal	CNP	CNP	BTH
	Slow path detection	Slow path	BTH
	RTT Probing	RTT Request	BTH, RTTReqETH
		RTT Response	BTH, RTTRespETH

5 Reliable Connection Service

Reliable Connection (RC) provides a guarantee that semantic packets are delivered from a Sender to a Receiver at most once without corruption, in the presence of lossy networks and multi-path transmission.

With out-of-order packet arrivals, lost packets must be quickly identified and retransmitted. To achieve this goal, key elements of the Reliable Connection service include a new PSN assignment method for the RDMA Read operation, an acknowledgment mechanism allowing the sender to ascertain that the semantic packets have been successfully delivered; packet and message numbering mechanisms to allow the sender to correlate acknowledgement packets with the semantic packets; heuristics in combination with packet numbering mechanism to detect missing packets under multi-path; and eventually a timer to allow detection of dropped or missing acknowledgment messages based on timeout.

Note that CRCs are checked at lower protocol layers and may result in packets being dropped before they are delivered to the transport layer.

To provide a clear understanding of the Reliable Connection service, this specification describes the endpoint behaviors from both the Sender-Receiver and Requester-Responder perspectives, as appropriate. Sender-Receiver perspective is about the sending and receiving of semantic packets and acknowledgements, while Requester-Responder perspective is about sending and receiving of requests and responses in terms of transport primitives.

5.1 PSN

Each packet is transmitted with a PSN in its BTH. PSNs are used to identify missing or out-of-order packets, and, for Reliable Connection service, to relate an acknowledge packet to a given semantic packet. A sender assigns PSNs to newly created semantic packets. The PSN value increases monotonically in one PSN space, and the initial value is negotiated during connection establishment. There is a relationship between the PSN on a requester's Send Queue and the PSN on the responder's corresponding QP (for Send, Write and Read Request primitives). Also, there is a relationship between the PSN of Read Response/AtomicAck on a responder and the PSN on the requester's corresponding QP (for Read Responses). Thus, a QP's outbound packets are in two separate PSN spaces, each for Send Queue and Read Response/AtomicAck. Accordingly, a QP's inbound packets are also in two separate PSN spaces. There is no relationship between the PSNs used by the Send queue and Read Response/AtomicAck of a QP, or between different connections.

There are two situations that must be detected and resolved at the receiver to ensure reliable operation: duplicate packets and out-of-order packets.

- Duplicate Semantic Packet

A duplicate packet may be recognized by the receiver if the sender injects a semantic packet into the fabric more than once. This occurs when the sender detects a condition for which the prescribed recovery mechanism is to retry the operation. There are two primary causes for the requester to inject duplicate semantic packets, both leading to a timeout. 1) Acknowledgement packets are lost or delayed in the fabric for a very long time; 2) A semantic packet is delayed in the fabric for a long time (but not lost) and there are no follow-up semantic packets, then a timeout will occur at the sender. Retransmitting the packets that are presumed lost will cause

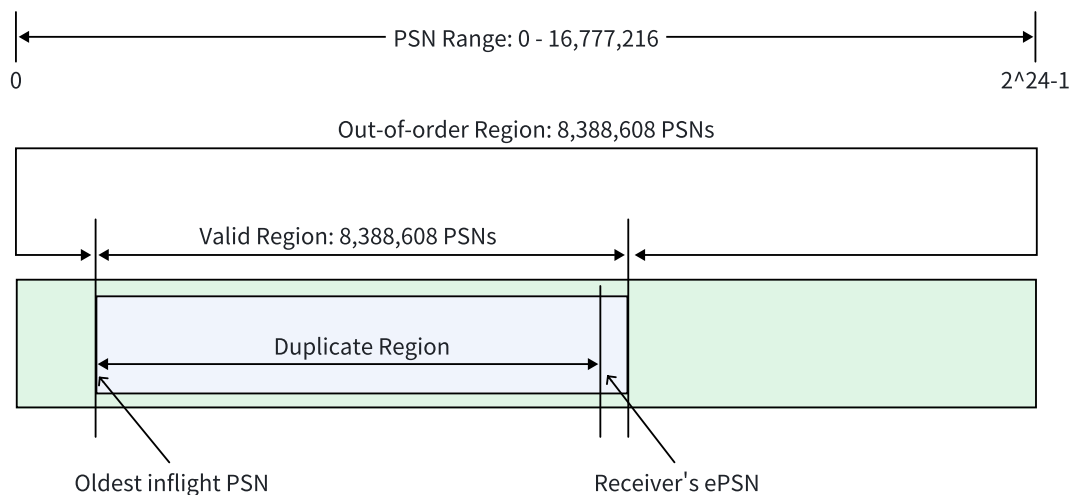
duplicate packets at the receiver, and the receiver will drop duplicate packets and schedule an ACK/SACK.

- Out-of-order Semantic Packet

This condition occurs when the receiver believes that one or more semantic packets have been reordered by the fabric, based on the non-contiguous PSNs in the received packets. Out-of-order packets should be accepted and processed independently by the receiver.

In order to enable the receiver to distinguish duplicate packets from out-of-order packets, a PSN space shall have a series of PSNs no greater than 8,388,608 (2^{23}) outstanding at any given time. Therefore, a PSN space shall have no more than 8,388,608 packets outstanding at any given time. Thus, the PSN space (consisting of a range of 16,777,216 PSNs) is divided into two regions called the valid region and the out-of-order region, each occupying a range of 8,388,608 PSNs.

Note that the length of the receiver's PSN bitmap is another practical factor that limits the out-of-order region. A receiver cannot record the arrival of a packet in its PSN bitmap if the packet's PSN is equal to or larger than $aPSN + \text{bitmap_length} + 1$, thus the packet is dropped by the receiver.



The receiver further divides the valid region into a singular expected PSN (ePSN) and a duplicate region. The receiver's ePSN is the next in-order PSN that the receiver expects to receive in the next new packet (i.e., $aPSN + 1$). The duplicate region is therefore defined to be the entire valid region, except for the single expected PSN. Hence, a duplicate PSN is a PSN falls in the duplicate region.

5.2 MSN

On the sender side, the MSN is a monotonically increasing sequence number assigned to messages, starting from 1; on the receiver side, MSN is a cumulatively acknowledged sequence number for messages. The messages include RDMA Read Request, RDMA Read Response, RDMA Write, and Send. For a requester, it assigns MSNs to RDMA Read Request, RDMA Write and Send, which are from the SQ. For a responder, it assigns MSNs to RDMA Read Response and AtomicAck, which bears a one-to-one mapping to the Read Requests/Atomic Requests buffered by the responder. The MSN assigned to a Response message is referred to as MSN_Rsp. A QP's

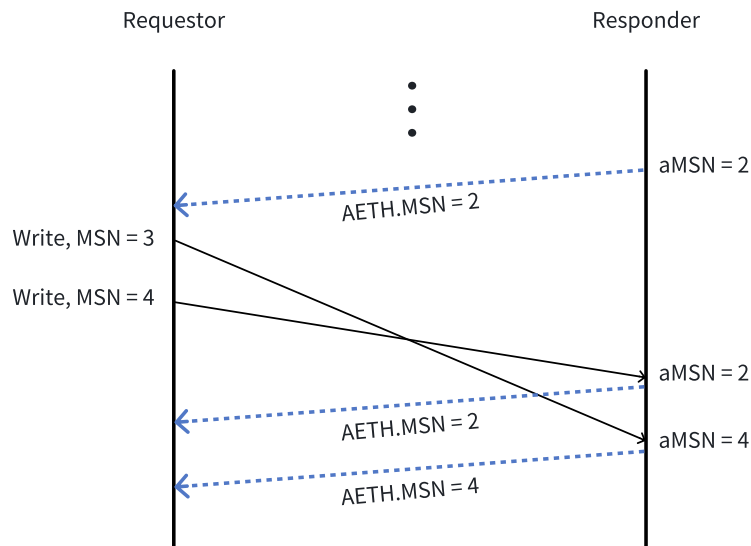
SQ and Read Response/AtomicAck have separate MSN spaces. There is no relationship between the MSN used by SQ and the MSN_Rsp used by Read Response/AtomicAck of a QP, or between different connections.

Each packet is transmitted with an accompanying MSN located in the MSNETH. For Request packets associated with RDMA Read, RDMA Write, and Send operations, the MSNETH.MSN field is populated with the MSNs corresponding to their respective messages. In the case of Read Response packets, the MSNETH.MSN field is populated with the MSN_Rsp. It is important to note that Read Response packets also incorporate an MSN field within the AETH. This field is populated with the MSN of the corresponding Read Request, enabling the requester to correlate a Read Response packet with an inflight Read Request. For acknowledgment packets (ack_pkts), the AETH.MSN field is populated with the aMSN on the receiver's end. It serves as an acknowledgment number, indicating that all messages with an MSN less than or equal to this number have been successfully received.

The MSN serves the following primary purposes.

1. It facilitates the WQE completion process at the sender. For operations among RDMA Read Request, Read Response, RDMA Write, and Send, the MSNs in their ack_pkts represent the cumulatively acknowledgement MSN (aMSN), which means messages with MSNs equal to or smaller than this value have all arrived at the receiver. If AETH.MSN advances, the increment corresponds to the number of messages that can be completed on the sender's end. However, should a Read operation be among the messages awaiting completion, only those messages preceding the Read operation can be completed. Any subsequent messages must await the completion of the Read operation, provided it has not already been completed. Furthermore, if an implementation has physically separate SQ and Read Response/AtomicAck, additional mechanisms are required to determine the number of messages that can be completed in each queue.
2. The MSN in AETH of the inbound Read Response packets is used to correlate a Read Response packet with its corresponding Read Request. This allows the requester to determine whether all response packets for a given Read Request have been received.

The receiver side maintains an aMSN. The figure below demonstrates the updating process of aMSN. Packet out-of-order may cause a message with a larger MSN arrives earlier than a message with a smaller MSN. In the example below, the single-packet Write message with an MSN of 4 arrives before the single-packet Write message with MSN 3. Upon receipt of Write 4, the aMSN at the receiver remains at 2. After the receipt of Write 3, the aMSN advances to 4. Consequently, an ACK with AETH.MSN = 4 (aMSN) is issued, enabling the sender to ascertain that all messages up to MSN 4 have been received.

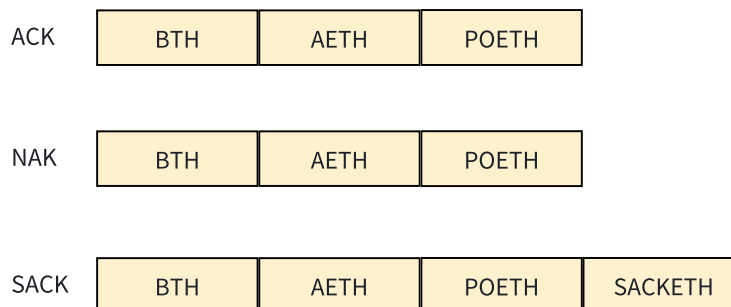


5.3 Acknowledgement Protocol

The Acknowledgement Protocol, plus packet sequence numbers, are fundamental components of the Reliable Connection service. This section describes the protocol, provides a set of rules governing generation of ACK, SACK and NAK packets, and specifies the Sender's required actions when it receives an ACK, SACK or NAK packet. While describing the protocol, the "Sender-Receiver" perspective is usually employed.

The Acknowledgment Protocol allows the sender to ascertain deterministically if the receiver has received the semantic packets. The ascertainment is achieved using ack_pkts, especially, the PSN and the bitmaps. In addition, the MSN field in the ack_pkts can also be leveraged. Since ack_pkt can get lost in the fabric, the Acknowledgement Protocol will not take effect. Thus, a sender is required to implement a Transport Timer as a supplement to the Acknowledgment Protocol. Packet retransmission happens when the timer expires. Transport Timer is defined in the same way as RoCEv2.

The formats of the acknowledgement packets are described below.



5.3.1 ACK/SACK Protocol

Each semantic packet must be acknowledged. ACK packets may be coalesced, allowing a single ACK to serve as confirmation for one or more previous semantic packets. When a receiver generates an ACK/SACK, its BTH.PSN field is populated with the acknowledgement PSN (aPSN). When a sender receives an ack_pkt, it should advance its aPSN to ack_pkt's BTH.PSN. For Read,

Atomic, and Send-with-Invalidation, the ACK packet acknowledges that the packet has been received. For other types of semantic packets, the ACK packet acknowledges that the packet has been received and its memory access has been validated.

A SACK packet is used to selectively acknowledge the out-of-order semantic packets at the receiver. The coalescing of SACK packets is implementation-specific; however, this specification encourages minimizing the number of SACK packets to conserve bandwidth, considering the 128-bit long bitmap. Therefore, instead of generating a SACK for each out-of-order packet, veRoCE recommends a "lazy SACK" scheme: only when the out-of-order degree (OOD) at the receiver side exceeds a threshold shall the receiver send out a SACK. This also means the receiver keeps returning ACKs when OOD is below threshold. The OOD is defined by the difference between the highest received PSN (hPSN) and the aPSN. The OOD threshold for sending SACK packets can be considered as the maximum out-of-order degree caused by multi-path. The receiver shall keep sending SACK packets until OOD goes below the threshold. The reasons why lazy SACK is recommended are: 1) SACK packets are generally more costly to generate than ACK packets. 2) Consecutive SACK packets often have overlapping bitmaps, providing limited incremental information. When the OOD is over the threshold, the SACK packets can be further coalesced.

Unlike ACK and NAK packets, which share the same Opcode, a SACK has its own distinct Opcode. Although it still includes an AETH, the AETH.syndrome field is exclusively used to convey flow control credits. A SACK packet also contains a SACKETH, which includes the 128-bit bitmap. If a sender receives a SACK packet while using "Lazy SACK", it can assume the aPSN + 1 is lost. Afterwards, it can decide to retransmit the aPSN + 1 along with some unreceived PSNs close to the it, as indicated in the bitmap. If multiple SACKs are received, an additional mechanism is required to prevent redundant retransmissions of the same PSNs triggered by each SACK. The length of receiver's physical Rx bitmap may be larger than 128-bit, depending on implementation. Since a SACK allows specifying the starting PSN of its bitmap, the receiver can optionally extend the bitmap beyond 128 bits by sending multiple SACKs.

The PO field is populated with the PO corresponding to aPSN + 1. This field helps quickly locate the data to be transmitted for a WQE, which can be identified by the AETH.MSN in ACK/-SACK. Specifically, SACK.PO is used for fast selective retransmission, while ACK.PO is used for RTO retransmission.

5.3.2 NAK Protocol

The veRoCE NAK protocol remains consistent with the IB specification. NAK Code b'00000 (PSN Sequence Error) is repurposed as Packet Drop NAK.

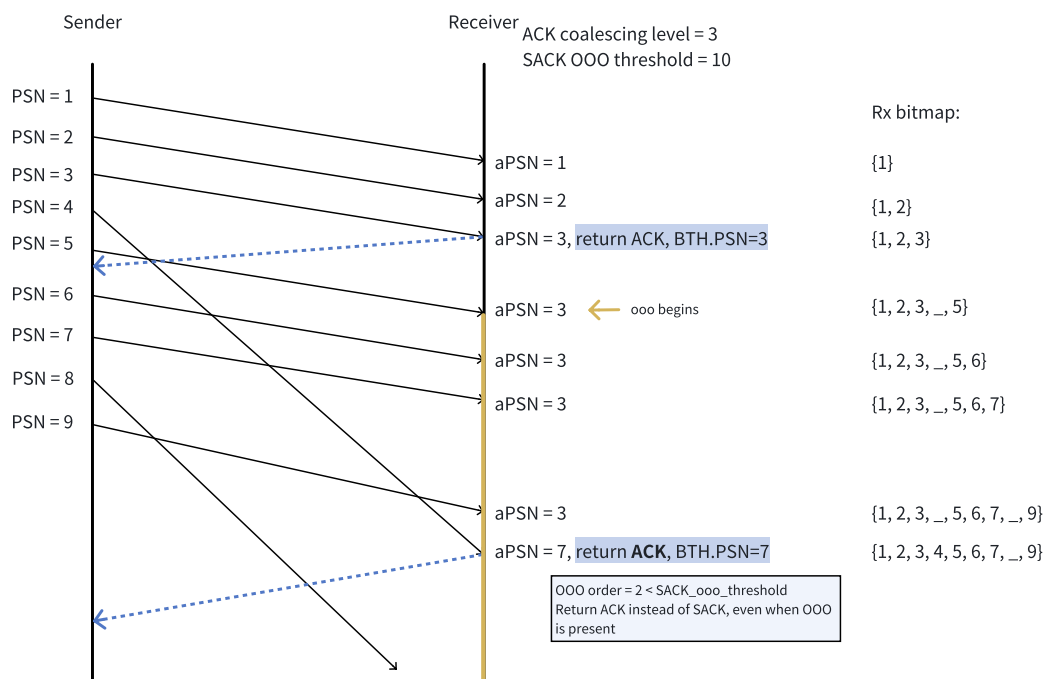
For Packet Drop NAKs, the NAK packet shall be transmitted immediately after the error is detected. The PSN field in BTH is populated with the PSN of the trimmed packet. The MSN field in AETH is populated with the MSN of trimmed packet. For all the other NAKs, when the error is detected, the receiver shall record the error but shall not transmit a NAK packet until all preceding packets have been received. If, during this interval, any such preceding packet is also found to be in error, the receiver shall update the recorded error to that packet. Consequently, the NAK shall report the error caused by the packet with the lowest PSN among the outstanding packets, and it shall be sent only after all packets with lower PSNs have been received. The values of PSN field in BTH and MSN field in AETH are consistent with RoCEv2: BTH.PSN is populated with the PSN of the error packet, which is aPSN plus one; AETH.MSN is populated with aMSN, which

equals the MSN of the error packet minus one. If there are preceding Read Request packets, the receiver is not required to wait for the corresponding Read Response packets before sending the NAK.

Moreover, if the responder encounters a remote access error or remote operation error while processing a Read Request, it returns a NAK packet with the appropriate NAK code instead of a Read Response.

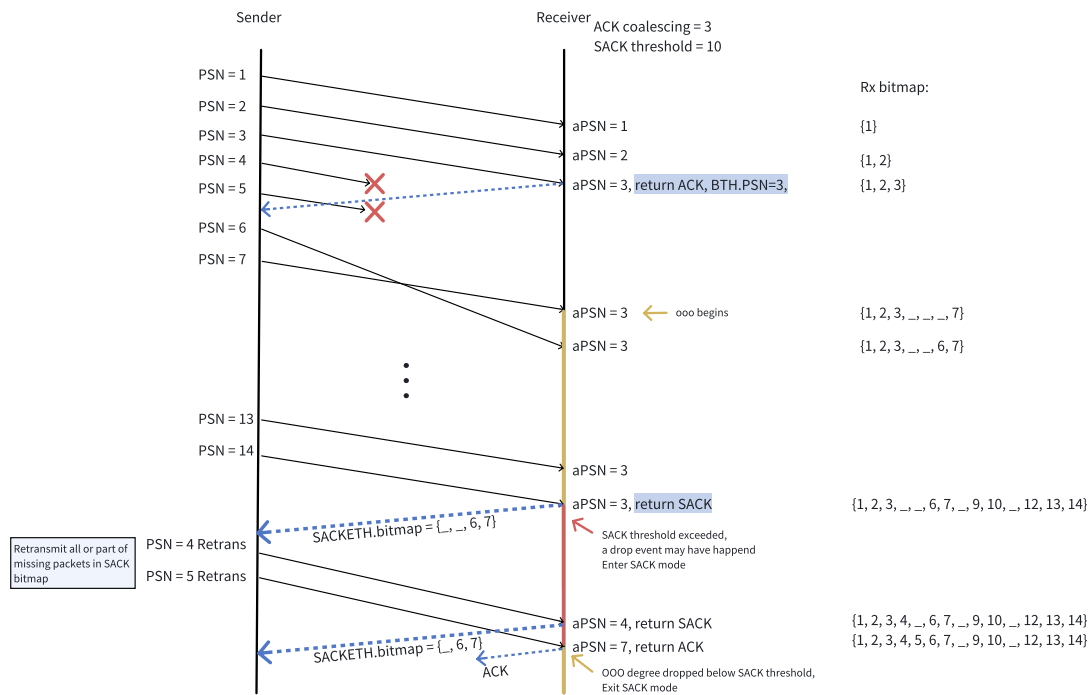
5.3.3 ACK Example

In Reliable Connection service, ack_pkts are allowed to be coalesced for multiple benefits, including saving bandwidth, saving NIC's PPS (packet per-second) capability. The coalescing rule is implementation specific. An example for ACK is illustrated below, where ACK coalescing is in effect. Due to network reordering, the aPSN may advance more than 1 when receiving a semantic packet. In this example, the reception of packet 3 triggers an ACK with BTH.PSN=3, which acknowledges 3 packets. Next, the reception of packet 4 (Rx bitmap is 1, 2, 3, _, 5, 6, 7) advances aPSN from 3 to 7. Therefore, the PSN field of the second ACK is set to 7.



5.3.4 SACK Example

The figure below illustrates an example of SACK generation, where the out-of-order degree (OOOD) threshold is configured to 10 at the receiver. Due to network issues, packets 4 and 5 are lost, which results in the receiver's aPSN remaining at 3 upon receipt of packet 7. The receiver refrains from issuing a SACK until the arrival of packet 14, at which point the OOOD surpasses the threshold. Upon receipt of the retransmitted packets 4 and 5, the receiver's OOOD reduces to 6, prompting the receiver to issue an ACK rather than a SACK for subsequent packets.



5.3.5 Packet Drop Detection

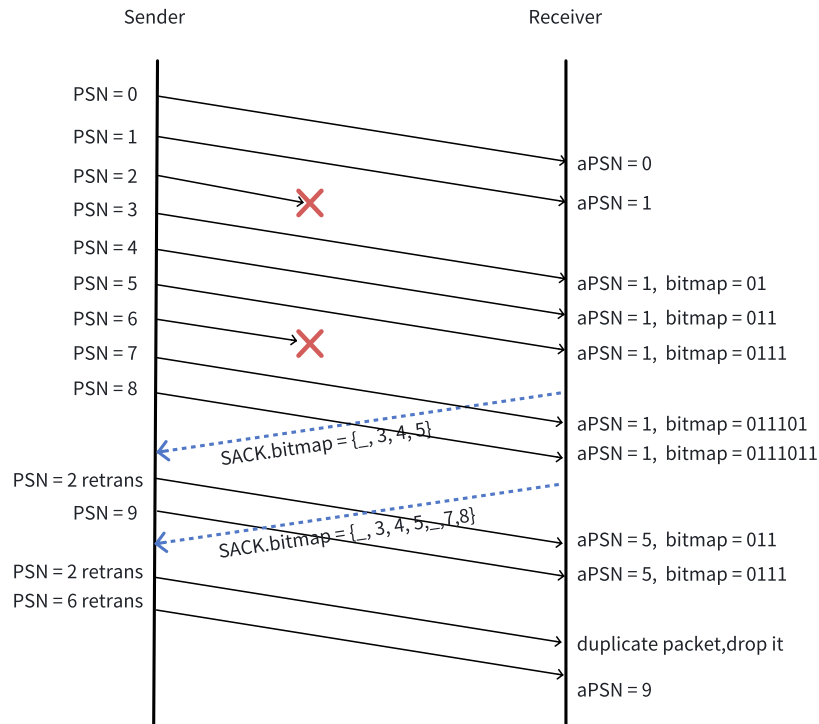
There are two methods to detect packet drops in a multi-path environment.

- **SACK packets:** The bitmap in SACK packets serves as a direct indicator of which PSNs have been received and which have not up to the current moment. Heuristics can be applied to infer dropped packets. When using the "Lazy SACK" method, the sender can assume that the aPSN + 1 has been lost upon receiving a SACK and subsequently initiates the retransmission of aPSN + 1, along with a certain number of un-received PSNs close to it.
- **Transport Timer:** A retransmission shall be triggered if the packet is not acknowledged before the Transport Timer expires (RTO). Once an RTO retransmission is triggered, the sender shall halt SACK-triggered fast retransmission until the aPSN + 1 is acknowledged.

In order to detect excessive retries, the sender shall maintain an error retry counter in the same manner as RoCEv2.

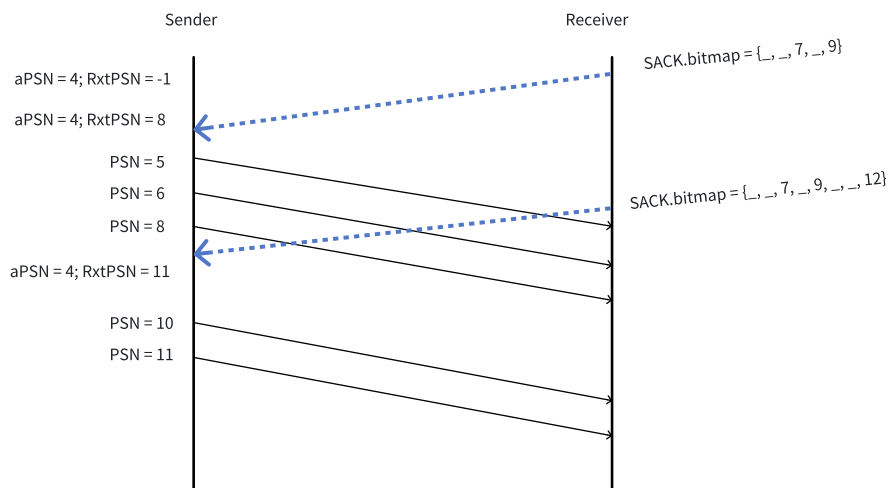
5.3.6 Fast Selective Retransmission

Consecutive SACK packets may have overlapping bitmaps. If every SACK packet simply triggers the retransmission of the un-received packets in the range [aPSN + 1, aPSN+N], there will be a lot of unnecessary retransmissions, as consecutive SACK packets are very possible to contain overlapped bitmaps. An example is provided in the figure below. Packet 2 is re-transmitted twice because the two SACK bitmaps indicate that Packet 2 has not been received.

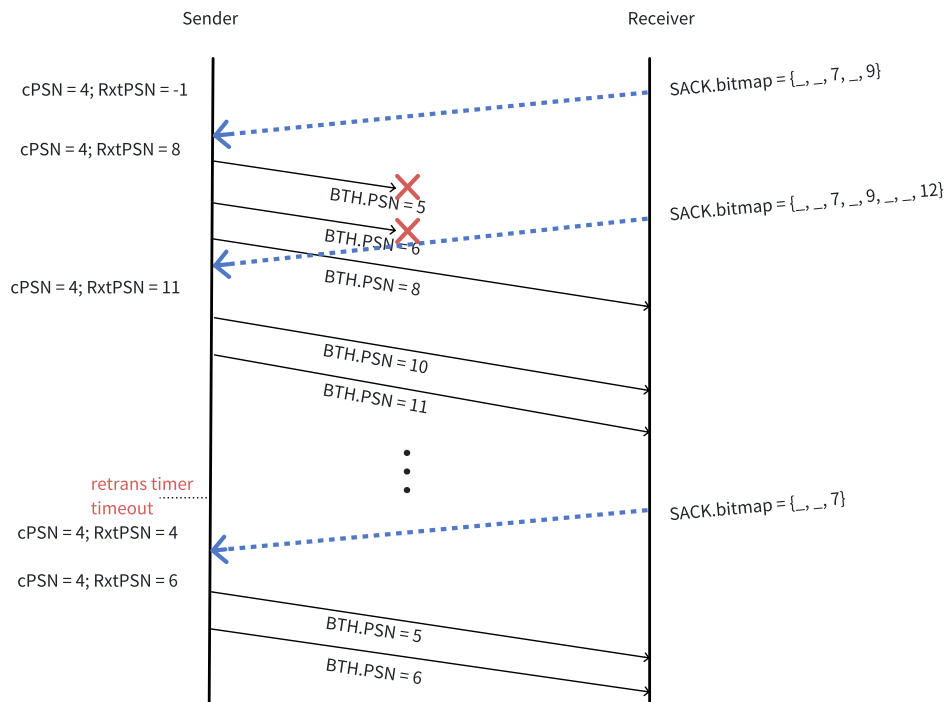


veRoCE recommends a Fast selective retransmission mechanism that alleviates unnecessary retransmissions. The sender or the receiver maintains a variable RxtPSN that records the highest valid PSN from the last SACK bitmap, which is updated for each SACK received or transmitted. For each new SACK received or transmitted, only bitmap entries with PSN > RxtPSN shall be considered for selective retransmission. To avoid the RTO caused by lost retransmitted packets, the sender or the receiver shall also record the time when RxtPSN is lastly updated. When RxtPSN is not updated for a certain period of time, RxtPSN shall be reset to aPSN to allow a second selective retransmission. A typical value for the time threshold is the network RTT.

The figure below depicts how fast selective retransmission works when RxtPSN is maintained at the sender. Note that such design is not mandated by veRoCE, as RxtPSN can also be maintained at the receiver. After the sender receives the first SACK, the sender parses the bitmap, retransmits packets [5, 6, 8] and updates RxtPSN to 8. When the second SACK arrives, the retransmitted packets [5, 6, 8] are still in the network. With the help of RxtPSN, the sender only retransmits packets 10 and 11, which are greater than RxtPSN.



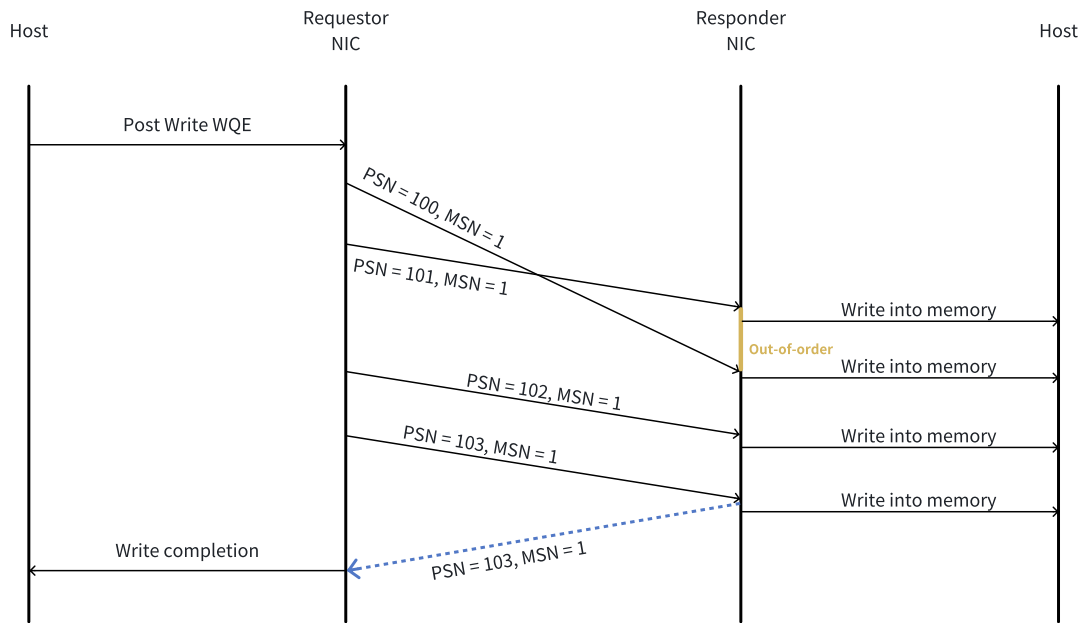
If the retransmitted packet gets lost again, RxtPSN will be stalled for a long time period, leading to the reset of RxtPSN. The figure below depicts this process. The first SACK in the figure triggers the retransmission of packets [5, 6, 8], but packets [5, 6] get lost again. The second SACK only retransmits the packets 10, 11, as RxtPSN has advanced to 8. When the third SACK arrives, the RxtPSN has been stalled long enough to reset RxtPSN to aPSN (i.e., 4), and packets [5, 6] are retransmitted for the second time.



5.4 Send/Write Operations

The RDMA Write operation enables a Requestor to write data into the virtual address space of a responder. The message size can range from zero to 2^{31} bytes (inclusive) for Reliable Connection. The data of a Write message is written to a contiguous segment within the virtual address space of the destination QP, which may not correspond to a contiguous range in physical memory. Each Write request packet carries a RETH, which contains the virtual address of the corresponding packet, to enable direct data delivery to remote memory.

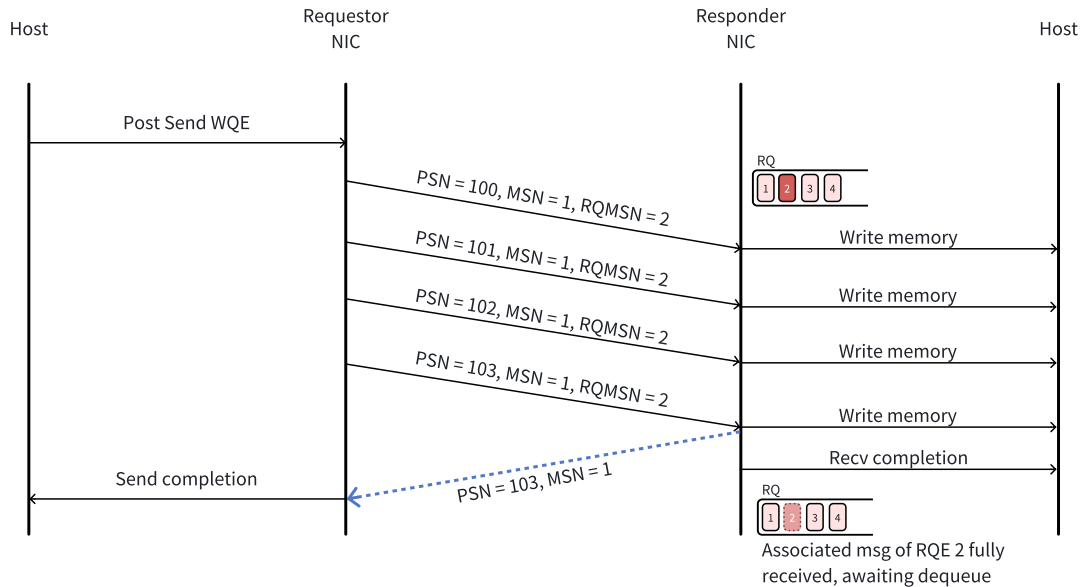
An example of the operation of RDMA Write is as follows:



1. The requestor posts a Write WQE to the Send Queue. The QP creates 4 write request packets (PSN = 100-103) based on the message size and MTU.
2. The requester transmits the packets sequentially to the responder. The packets may get reordered by the network. For example, packet 100 and packet 101 arrive out-of-order at the responder. The responder directly places the data to host memory without reordering them.
3. The responder returns ack_pkts to the requestor to acknowledge the arrival of the Write requests, and the responder may perform ACK coalescing to reduce the number ack_pkts. For example, ACKs of the last 4 packets are coalesced. After the straggler packet of this message is processed by the responder, the responder replies an ACK with AETH.MSN set to 1. The straggler packet of a Write message can be identified by the PSN bitmap on the responder.
4. Upon receiving the ack_pkt with the new MSN value, i.e., 1 in this example, the requestor completes the write message.

The Send operation enables a requestor to transfer data to a responder without knowing the memory location on the responder side. The responder places the data into the available receive buffer for that QP pointed to by a WQE in the QP's Receive Queue. The Receive WQE for a Send message is explicitly specified by the RQETH.RQMSN field of every Send Request packet, so that each Send Request can still find its corresponding WQE when they arrive at the responder out-of-order. A Send message may range in size from 0 to 2^{31} bytes for Reliable Connection. A message larger than MTU shall be segmented into multiple MTU-sized packets, possibly except the last packet.

If a Send message wants to carry Immediate Data, the Immediate Data is attached to the last or only packet of this message. The Opcode of this packet is set to "Send-with-ImmDt".



Once a Send message is fully received, the responder is tasked with removing the corresponding RQE from the RQ and inserting a CQE into the CQ. However, given that request packets can arrive in a non-sequential manner, this operation is contingent upon all preceding RQEs having their messages fully received. If this condition is not met, the responder must delay the operation until the requirement is satisfied.

5.4.1 Requester: Generating Request Packets

5.4.1.1 Generating PSN For Reliable Connection service, the requester must place its current PSN value in the BTH.PSN field of every request packet. During connection establishment, the transport layer's client programs the next PSN to any value between zero and 16,777,215. The initial ePSN value on the responder side must be loaded with the same value. The requester shall increment the PSN value by one for each request packet it generates.

5.4.1.2 Generating MSN and PO A Requester assigns curMSN to a Send/Write WQE when it is scheduled out from the SQ. The curMSN increases monotonically for Send/Write WQEs as well as Read Requests. A Requestor using Reliable Connection service shall initialize the curMSN to 1. One SQ WQE may create one or more request packets; especially, a Read Request creates exactly one packet.

The position of a packet within its message is referred to as the In-message Packet Order, and it is carried by each request in the POETH.PO field. The Requester calculates this value for a request packet by subtracting the starting PSN of the message from the packet's PSN, so the In-message Packet Order is in range $[0, \lceil msg_len/MTU \rceil - 1]$. The PO is returned to the sender and is used to quickly locate the retransmission data within a WQE.

5.4.1.3 Supporting Direct Data Placement (DDP) To support DDP for Write operations, each packet in a Write message must carry an RETH, which specifies the starting address of the target memory for its payload. For Write-with-Imm messages, since they consume RQEs in the responder's Receive Queue (RQ), the last packet must also include an RQETH header, with the RQMSN field indicating the sequence number of the corresponding RQE. Similarly, Send and Send-with-Imm messages, which also consume RQEs in the responder's RQ, must include the

RQETH header. Additionally, Send and Send-with-Imm messages must carry the POETH header, where the PO field indicates the position of the packet within the message. The combination of MSN and PO can be used to further assist DDP processing.

5.4.2 Responder: Receiving Request Packets

5.4.2.1 Recording PSN in bitmap The inbound Request packets shall first go through a validation process to make sure that they are valid. Afterwards, their PSN's are recorded in a PSN bitmap, in which each bit represents a packet's arrival or not. The starting PSN of the packet PSN bitmap is $aPSN + 1$, and the end of it is $aPSN$ plus its length. The length of the bitmap is implementation specific, it is associated with the resource of the responder. Arriving request packets with PSN larger than the bitmap's end PSN shall be dropped silently. And arrival request packets with PSN in the duplicate PSN range should be dropped and the receiver should generate an acknowledge packet.

5.4.2.2 Locating RQE For Send and Write-with-ImmDt request packets, the responder is required to identify their corresponding RQE, which contains the address for the data placement. The responder must locate the RQE by matching the RQETH.RQMSN with the sequence number in the Receive Queue. If the number of RQE is insufficient, the responder is obliged to discard the packet and respond with an RNR-NAK. The sender deals with RNR-NAKs in the same way as RoCEv2.

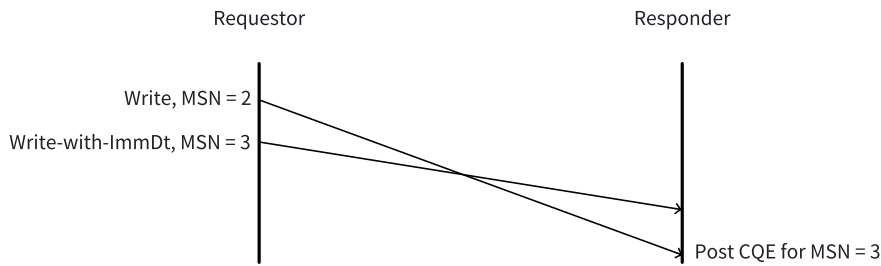
5.4.3 Responder: Generating Acknowledgement Packets

The `ack_pkts` are used to inform the requester of the arrival of the request packets. The PSN field is populated with $aPSN$, and the MSN field of AETH is populated with $aMSN$. Under multi-path, it is normal that request packets can arrive out-of-order, and SACK packets must be used to selectively acknowledge the out-of-order packets. Instead of generating SACK packets whenever there is an out-of-order request packet, a lazy SACK scheme can be used to conserve bandwidth (refer to Sec. 5.3.1 for details).

5.4.4 Responder: Message Completion

For Send, Send-with-ImmDt and Write-with-ImmDt messages that consume RQEs, the responder is required to retire the associated RQE and post a CQE to the CQ if the message itself, as well as all the preceding messages, have been fully received. For instance, as depicted in the figure below, the responder posts the CQE for the Write-with-ImmDt request with an MSN of 3 after the Write request with an MSN of 2 has been fully received.

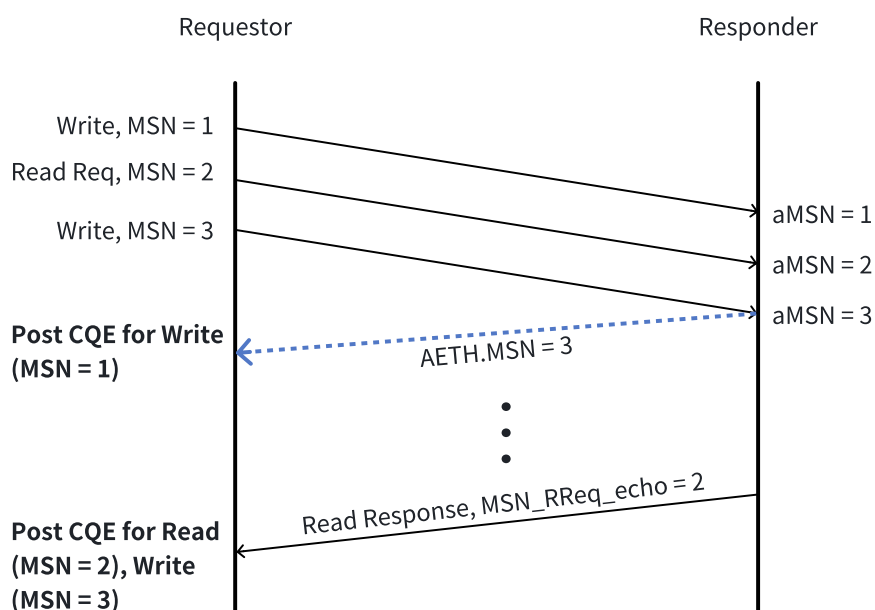
For Write-with-ImmDt and Send messages, the immediate data carried by the last packet needs to be copied to the CQE. With out-of-order arrival, the last packet may arrive before the CQE is generated and immediate data needs to be cached on the responder. A suggested location for the immediate data is the associated RQE.



5.4.5 Requester: Message Completion

The requester can determine that messages with a MSN value smaller than or equal to the aMSN returned by ack_pkts have been completed. Consequently, an increase in aMSN from new ack_pkts means the aggregate number of WQEs that the requester can retire from the QP's SQ. However, when there are outstanding Read Request WQEs in the SQ, the situation becomes more nuanced. In this case, the requester only completes outstanding WQEs up to either the first outstanding RDMA READ request, or the WQE whose MSN matches the new aMSN, whichever comes first. This is due to the fact that an advancement in aMSN only ascertains the arrival of the Read Request packets at the responder, but does not guarantee that all the Read Response packets for these Read Requests have returned to the requester. Therefore, when a Read Request WQE is encountered during the completion process, the requester must verify that this Read Request has been fully responded before marking it as completed.

The figure below illustrates an example of this scenario. Initially, there are three outstanding messages: a Write message, a Read Request message, and another Write message. Subsequently, a coalesced ACK with an MSN of 3 returns to the requester before the Read Response packets. The requester's completion process is required to halt at the Read Request. Once all the Read Responses have been received, the requester can complete the remaining two messages. (For brevity of this example, the Read Request is depicted with only a single response packet.)



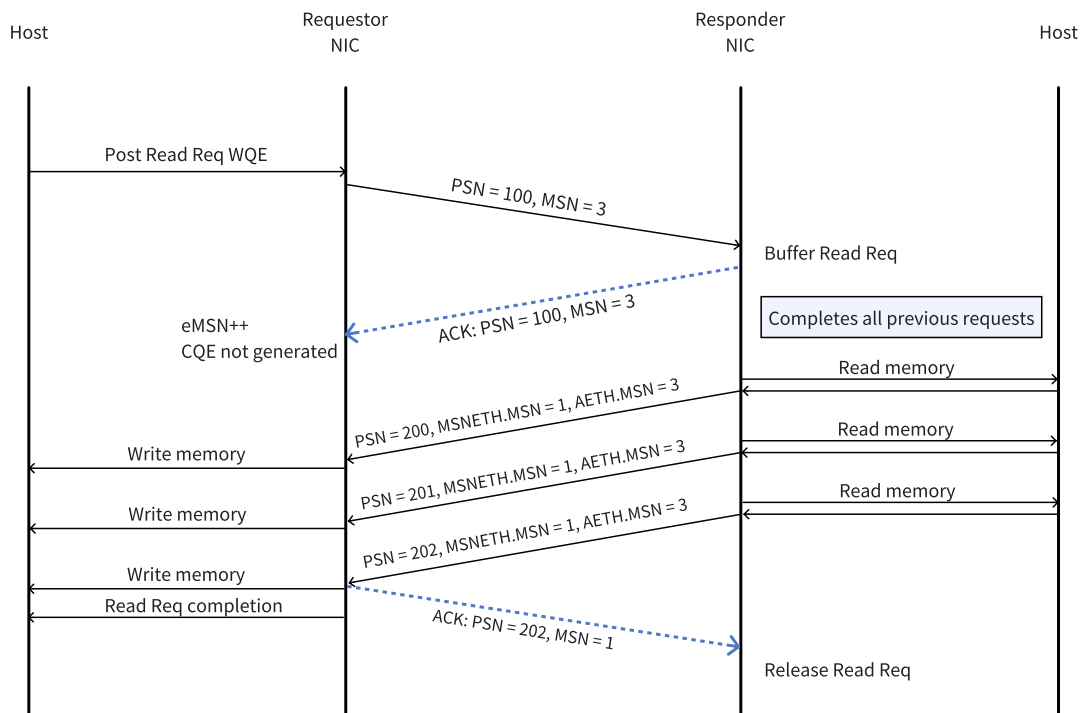
A potential method to manage the complex message completion process described above,

particularly when outstanding Read Request messages exist, involves generating CQEs as soon as the Read Requests receive packet-level acknowledgment. However, these CQEs are initially marked with an "incomplete" status code. This status code is updated to "complete" once all response packets for a particular Read Request have been received. If all the Read Response packets for a Read Request are received before the ack_pkts for that Read Request, the requester should use a 1-bit flag in the Read Request WQE to signify that this Read Request has been fully responded to, and directly generate a "complete" CQE for it.

5.5 Read Operation

RDMA READ operations enable a requester to read a virtually contiguous block of memory on a responder. A single RDMA READ request can read from zero to 2^{31} bytes (inclusive) of data for Reliable Connection. The Read Request occupies only a single PSN. Subsequent requests use the next PSN sequentially without skipping any PSNs. After receiving the Read Requests, the responding side buffers them until all the prior requests are processed. For a Read Response, its AETH.MSN is populated with the MSN of the corresponding Read Request.

An example of the operation of RDMA Read is as follows:

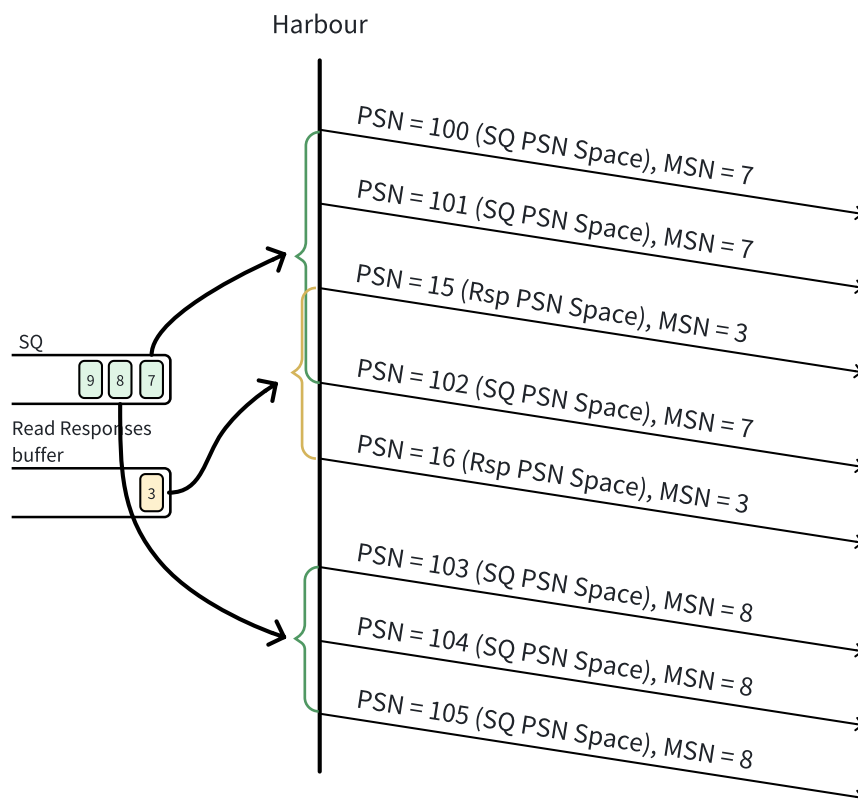


1. The requestor posts a Read Request WQE to the Send Queue, and a Read Request packet is transmitted to the responder afterwards. The Read Request only occupies one PSN.
2. Upon receiving the Read Request, the responder shall reply an ACK to the requestor (also subject to ACK coalescing). This ACK does not signal the completion of the read request, but is an ascertainment that the Read Request has been received by the responder.
3. The Responder buffers the read request. When all the prior requests are processed, the responder starts generating a Read Response message for the Read Request.
4. The responder sends out Read Response packets as instructed by the buffered Read Requests. For one Read Request, the responder may generate multiple Read Response pack-

ets based on the message size and MTU. The packets from the Read Responses and packets from the SQ are in separate PSN spaces (illustrated in the figure below).

5. The requestor receives the Read Response packets and replies ack_pkts to acknowledge their arrival. When the received Read Response packets fill some "holes" in the PSN range, the PSN and MSN may be able to advance, and they are returned to the responder via acknowledgement packets.
6. The responder receives the ack_pkts. When the MSN in the ack_pkts has advanced compared with the last received MSN, the responder releases a number of Read Requests.

It's worth pointing out that a QP's outbound packets are in two separate PSN spaces, each for SQ and Read Response/AtomicAck. There is no relationship between the PSNs used by the Send queue and Read Response/AtomicAck of a QP, or between different connections. Similarly, there is no relationship between the MSN used by a QP's SQ and MSN used by its Read Response/AtomicAck.



5.5.1 Requestor: Generating Read Request Packet

A requestor generates a Read Request packet following the same process of generating a send/write request packet. The Read request only occupies one PSN. Subsequent requests will sequentially use the next PSN without skipping any PSN.

5.5.2 Responder: Receiving Read Request Packet

The responder generates ack_pkts for Read Requests once the packet is received. The responder can perform optional validation on the VA and R_Key before generating ack_pkts, to allow earlier error detection. Meanwhile, the responder buffers the Read Requests.

5.5.3 Responder: Generating Read Response Packet

When all prior messages to a Read Request complete, the responder starts processing the Read Request. The Read Request shall first go through a second validation process to make sure that they are valid. This additional validation is necessary because changes occurring during the buffering period, such as an MR deregistration, may invalidate a previously valid Read Request.

After validation process, the responder generates Read Response packets and assigns them PSNs from Read Response/AtomicAck PSN space, and assigns them MSNs from Read Response/AtomicAck MSN space. The AETH.MSN of a Read Response packet is populated with the MSN of its associated Read Request, and the AETH.MSN in ack_pkts is populated with the aMSN.

If an error occurs when processing a Read Request, the responder generates a NAK packet instead of Read Response packets, even if an ACK has already been generated for the Read Request. The BTH.PSN of the NAK packet is populated with the PSN of the associated Read Request, and the AETH.MSN is populated with the aMSN.

5.5.4 Responder: Supporting Read Response DDP

To support DDP processing of the Read Response payload at the requester, a responder must embed a POETH header in each Read Response packet, where the PO field indicates the position of the packet within the message.

5.5.5 Requester: Receiving Read Response Packets

The inbound Read Response packets shall first go through a validation process to make sure that they are valid. Then the requester records the PSNs of Read Responses in a separate PSN bitmap for Read Responses. The requester must use the AETH.MSN to identify the associated Read Request WQE in the SQ, which contains the target address for the Read Response message. Then the POETH.PO must be used to determine the address offset to place the payload of a Read Response packet.

5.5.6 Requester: Generating Acknowledgement Packets & Message Completion

The ack_pkts for Read Response packets uses ACK_Rsp Opcode, to be distinguished from the ACK Opcode. Their MSN and PSN fields for Read Response ack_pkts are populated with the aMSN and aPSN in a way similar to how the responder receives Send/Write request packets.

The requester also verifies if a Read Request's response packets have all been received, which may resume the SQ completion process. The requester shall maintain an additional timer for outstanding Read Requests. If a Read Request is not completed before the timer expires, the QP shall enter error state.

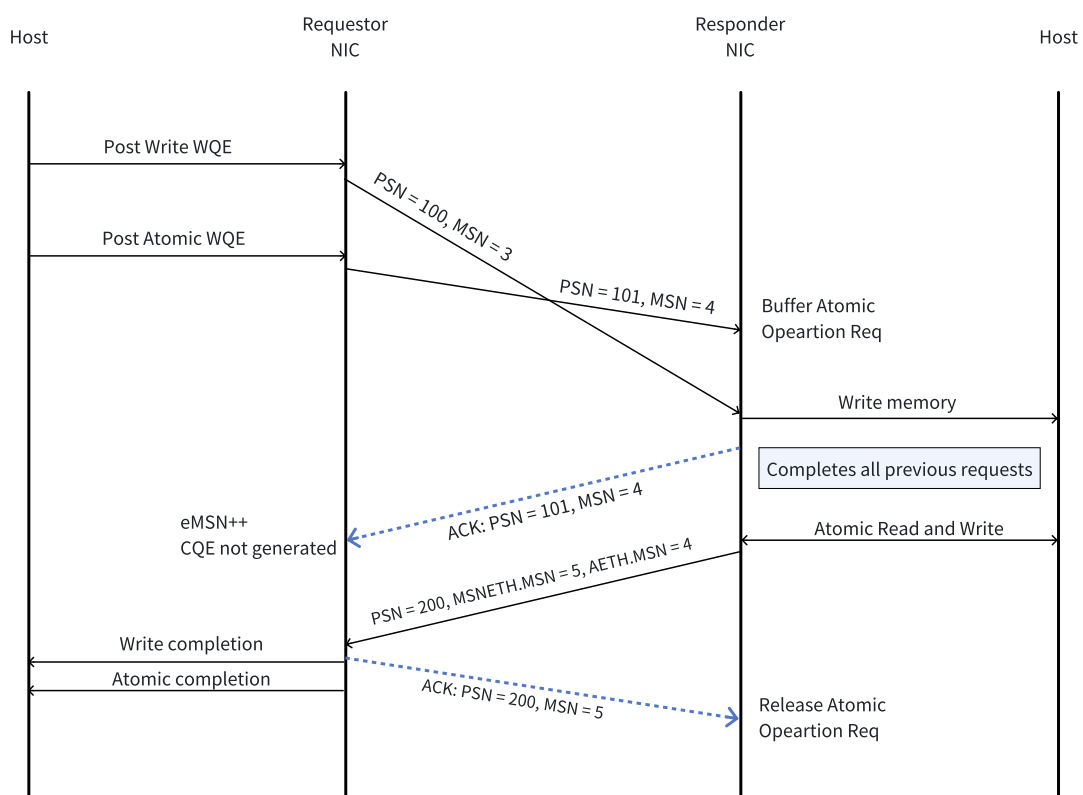
5.5.7 Responder: Receiving Acknowledgement Packets & Releasing buffered Read Request

The responder can determine that Read Response messages with a MSN smaller than or equal to the aMSN returned by ack_pkts have been completed.

5.6 Atomic Operation

Atomic Operations enables a requestor to execute a 64-bit operation at a specified address on a responder. The operations atomically read, modify and write the destination address and guarantee that operations on this address by other QPs do not occur between the read and the write. veRoCE supports FetchAdd and CmpSwap Atomic operations defined in IB Specification (Chapter 9.4.5). The Atomic operation request occupies only a single PSN. After receiving the Atomic Operation Requests, the responding side buffers them until all the prior requests are processed. The responding side may reply an ACK or SACK to indicate the reception of the Atomic Operation Request. However, it cannot be used as the completion signal for the operation at the requestor. The Atomic operation can only be considered complete when AtomicACK is received.

An example of the operation of Atomic is as follows:



1. The requestor posts an Atomic Operation Request WQE to the Send Queue, while there is an outstanding Write Request WQE. An Atomic Operation Request packet is transmitted to the responder afterwards. The Atomic Operation Request only occupies one PSN.
2. Upon receiving the Atomic Operation Request, the responder shall reply an ACK to the requestor (also subject to ACK coalescing). This ACK does not signal the completion of the Atomic operation, but is an ascertainment that the Atomic Operation Request has been received by the responder.
3. The Responder buffers the Atomic Operation request. When all the prior requests are processed, the responder executes the Atomic operation and generates an AtomicAck message for the Atomic operation.
4. The responder sends out AtomicAck messages. Each AtomicAck message occupies one

PSN and one MSN.

5. The requestor receives the AtomicAck packets and replies ACK_Rsp to acknowledge their arrival. When the received AtomicAck packets fill some "holes" in the PSN range, the PSN and MSN may be able to advance, and they are returned to the responder via acknowledgement packets.
6. The responder receives the ack_pkts. When the MSN in the ack_pkts has advanced compared with the last received MSN, the responder releases a number of Atomic Operation Requests.

5.7 Operation Ordering

The table below shows the ordering semantics for WRs submitted to the SQ. # means the order of WRs submitted to the SQ is always maintained at the responder hardware. F means order is maintained only if the second operation has Fence Indicator set. The ordering semantic generally follows the definition in IB Specification with a modification: a Send or RDMA Write will not necessarily complete before subsequent Sends or RDMA Write requests. If two Sends or RDMA Writes write to the same target memory, the data from the second operation may be overwritten by the first operation. This modification is to accelerate the DDP process and to avoid buffering data payload in the hardware. When a Send or RDMA Write request is posted to SQ when a previous request writing to the same memory region is outstanding, it is recommended to set the Fence Indicator to the second request to guarantee correctness of the results. The remote invalidate maintains the same ordering rules as RoCEv2. This means the remote invalidation operation in a Send with Invalidate shall be executed when all prior Send and Write operations complete.

Second First	Send	Remote Invalidate	RDMA WRITE	RDMA READ	Atomic
Send	F	#	F	#	#
Remote Invalidate	F	#	F	F	F
RDMA WRITE	F	#	F	#	#
RDMA READ	F	#	F	#	F
Atomic	F	#	F	#	#

5.8 Shared Received Queue

veRoCE supports Shared Received Queue defined in IB Specification (Chapter 10), including the ordering rule for Work Requests and Work Completions defined in Chapter 10.8.3.2. When a new Write-with-ImmDt/Send message arrives at a QP associated with an SRQ, the next available RQE in the SRQ is used to receive the message, even if the message arrives out-of-order. And the responder needs to maintain the association between the new message and the RQE it consumes. For new packets that belong to Write-with-ImmDt/Send messages that are not fully received, the responder needs to obtain their associated RQEs and perform DDP of the payload. For Receive WRs submitted through the SRQ, Receive Work completions on an QP must be returned in the order the Send WRs were submitted on the remote SQ.

5.9 Packet Trimming

Packet trimming can only work when both the endpoint and switch support this feature, like ECN/RED. Firstly, it needs to be enabled for queues on the switches. The semantic packets that wish to make use of packet trimming must be mapped onto those queues. Then, a trim codepoint on those packets must be set to "trimmable" by the sender, indicating that the packet is subject to trimming. The trim codepoint may reside in Ethernet header, IP header (e.g., the DSCP field), UDP header, or veRoCE BTH. Acknowledgement packets, congestion control signal packets, and RTT probing packets (including the RTT probings piggybacked by data packets) are not subject to trimming, so the trim codepoint on these packets must be set to "non-trimmable".

The switch must also expose two "trimming position" parameters, indicating the starting and ending positions of a packet that can be trimmed. The preserved parts after trimming must contain the necessary fields to quickly assemble a retransmission packet for a sender. After trimming, the ICRC and FCS must be recalculated and updated, to make sure that the fields in the trimmed packet can be trusted (This also implies that the ICRC and FCS fields must be preserved). The trim codepoint must be set to "trimmed" for the trimmed packet. Trimmed packets cannot be trimmed again by switches. It is up to implementation whether trimmed packets are put into another high-priority queue. If an implementation decides to do so, it may cause packets out-of-order since high-priority queues are usually empty and trimmed packets can arrive at the receiver earlier. But since veRoCE natively supports out-of-order delivery, special attention to this case is not required.

Upon receiving a packet with trimmed flag, the receiver must reply a Packet Drop NAK (AETH.syndrome=b'01100000). The BTH.PSN shall be filled with the PSN of the trimmed packet. Each Packet Drop NAK shall trigger a retransmission of a single PSN indicated by the NAK.

6 Error Handling

veRoCE inherits most error-handling logic from RoCEv2 but modifies key behaviors to support lossy networks and out-of-order delivery. The changes focus on removing obsolete errors (irrelevant to veRoCE's design) and adding new errors for Read Response and AtomicACK.

6.1 Removed Error Behaviors

6.1.1 Requestor-Side Removed Errors

The following errors are removed from Requestor side error behavior to support out-of-order delivery:

- "Packet sequence error. Retry limit not exceeded."
- "Packet sequence error. Retry limit exceeded."

The following errors are removed from the Requestor side error behavior, since ACK only signals the delivery of Read and Atomic requests:

- "Implied NAK sequence error. Retry limit not exceeded."
- "Implied NAK sequence error. Retry limit exceeded."

6.1.2 Responder-Side Removed Errors

The following errors are removed from Responder side error behavior to support out-of-order delivery:

- "Out of Sequence Request Packet."
- "Out of Sequence OpCode, current packet is 'first' or 'Only'."
- "Out of Sequence OpCode, current packet is not 'first' or 'Only'."

6.2 Added Error Behaviours

6.2.1 Requestor-Side Added Behaviours

The following error is added to Requestor side error behavior to support packet drop NAK:

Error	Description	Syndrome	Responder Fault Behavior Class
Packet Drop error.	Responder detected a trimmed packet. Requester may retry the request.	NAK – Packet Drop	Class A

6.2.2 Responder-Side Added Behaviors

veRoCE adds Responder-side error handling for Read Response and AtomicAck – these errors are triggered by NAK_Rsp from the Requester. A new fault behavior class is added to support responder-side recoverable errors:

Fault Behavior Class	NAK Codes Returned	Current Receiver WQE	Subsequent Receive WQEs	Final Receive Queue State
Responder Class N	Packet Drop NAK_Rsp	No WQE consumed	No impact	No change

The following errors are added to Responder side error behavior to handle Read Response and AtomicAck issues:

Error	Description	Syndrome	Responder Fault Behavior Class
Packet Drop error.	Requester detected a trimmed packet. Responder may retry the request.	NAK_Rsp – Packet Drop	Class N
Local Ack Timeout error. Retry limit not exceeded.	No ACK_Rsp from requester within timer interval. Responder may retry the request.	Locally detected error	Class N
Local Ack Timeout error. Retry limit exceeded.	No ACK_Rsp within timer interval. The responder performed retries, but all attempts failed.	Local detected error	Class C
Remote Operation Error	Requester encountered an error, (local to the requester), which prevented it from completing the Read Response or AtomicAck.	NAK_Rsp – Remote Operation Error	Class C
Unsupported OpCode.	Requester detected an unsupported OpCode.	NAK_Rsp – Invalid Request	Class C

7 Congestion Control

7.1 Congestion Notification

Each receiver QP maintains one or more Congestion Signal Contexts (CSC) to record congestion related information. There are two methods to echo the congestion signal back to sender. 1) Inband: ack_pkts can carry congestion signals like ECN (using BECN field in the BTH header), which could be utilized by window-based congestion control algorithms. 2) Out-of-band: the congestion signals can also be returned to the sender using standalone CNPs, enabling rate-based congestion control algorithms. An implementation must provide at least one of these mechanisms.

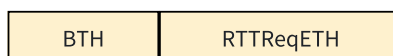
The generation rate of CNPs is implementation-specific. For example, while the receiver QP requires the NIC hardware to generate a CNP for each ECN-marked packet, the hardware may coalesce CNPs in a manner similar to ACK coalescing. Once a CNP is sent, the congestion signal context shall be reset.

7.2 RTT Probing

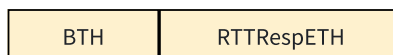
RTT is measured using standalone RTT probing packets. An RTT probe request returns 4 timestamps:

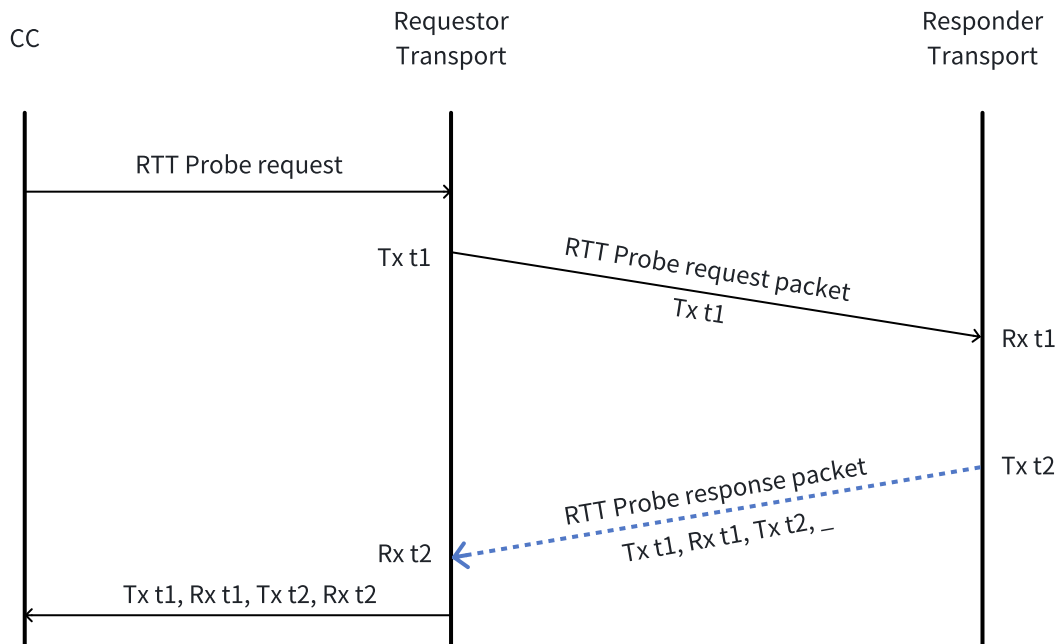
1. **Tx timestamp 1:** The RTT request packet Tx time at sender. This timestamp is carried by RTTReqETH in the request and echoed back by RTTRspETH in the RTT response.
2. **Rx timestamp 1:** The RTT request packet Rx time at receiver. This timestamp is carried by RTTRspETH in the RTT response.
3. **Tx timestamp 2:** The RTT response packet Tx time at receiver. This timestamp is carried by RTTRspETH in the RTT response.
4. **Rx timestamp 2:** The RTT response packet Rx time at sender. This timestamp is annotated by the sender.

RTT Probe Request



RTT Probe Response





The 4 timestamps are passed to the CC module. How these timestamps are used is determined by the CC algorithms. Three possible ways include:

- The network round-trip latency can be calculated by $Rx\ t_2 - Tx\ t_1 - (Tx\ t_2 - Rx\ t_1)$.
- The host processing latency can be calculated by $Tx\ t_2 - Rx\ t_1$.
- If the requestor and responder are time synchronized, the network one-way latency may be calculated by $Rx\ t_1 - Tx\ t_1$ and $Rx\ t_2 - Tx\ t_2$.

The UDP src port field of an RTT request is set based on which path it is probing, while the src port of the RTT response remains the same as its corresponding RTT request.

7.3 Congestion Control Modes

The congestion control mechanism for a Reliable Connection that employs multi-path can operate in two modes: connection-wise and path-wise. In either mode, the sender QP maintains one or more Congestion Control Context (CCC) that controls the sending rate. Note that for single-path Reliable Connection, there is no need to distinguish between connection-wise and path-wise mode. They are the same.

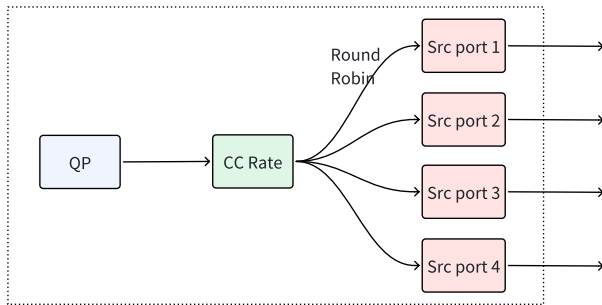
For sender-spreading multi-path transmission, the sender QP assigns different src ports to semantic packets, and the switch ECMP hashes the packets onto different paths. In the path-wise mode, the sender QP uses a separate sending rate on each path, and the receiver QP echos path-wise congestion signal back to sender via CNPs. The CNPs for different paths are distinguished by their UDP src ports associated with each path. The sending rate for each path can be determined either by an individual CCC dedicated to that path or by a single connection-wise CCC that allocates rates across paths using a weighted round-robin scheme.

In the connection-wise mode, the receiver QP does not discriminate the CNP from different paths. The number of ECN-marked packets from all the paths are cumulated onto one variable. The sender QP uses only one sending rate on all paths. That is, the sender QP only calculates one transmission rate, and it is evenly distributed on all the paths. This rate is expected to converge

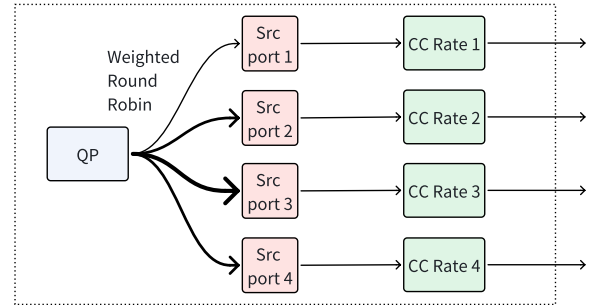
to the lowest available rate among all the paths. In both path-wise mode and connection-wise mode, the RTT probing is performed on a per-path basis. This helps the requestor to identify slow paths in the connection-wise mode.

For multi-path achieved by switch Adaptive Routing, the sender QP is unaware of the multi-path, and the receiver QP can not distinguish the packets from any path, therefore, connection-wise mode applies.

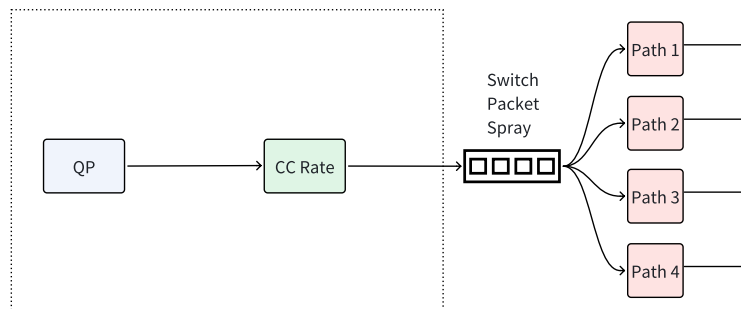
Connection-wise CC



Path-wise CC



Packet Spray



It is possible that different connections use different DSCP values to make use of the QoS policy on NICs and switches (while the packets within a connection use only one DSCP, regardless of whether the packets are transmitted over multi-path). veRoCE allows connections with different DSCP values to have different CC parameter values.

7.4 Slow Path Detection

Over time, path quality may degrade (e.g., due to link errors), turning a path into a “slow path” (high latency/loss). Slow path detection is an optional but recommended enhancement to improve network efficiency. Senders can use the following mechanism to identify and migrate traffic away from slow paths:

- Large PSN difference (on the receiver): if the difference between a new semantic packet's PSN and hPSN exceeds a threshold, then the packet is considered to be a slow packet. The receiver then sends a Slow-Packet Signal (a packet with Opcode bin10000100) to the sender. If the sender receives multiple Slow-Packet Signals for a given path during an interval, the path could be marked as a slow path.

The slow path detection algorithm can be implementation-specific. Other metrics, e.g., delayed ACKs, RTT probing/response loss, can also be used to identify slow paths.

8 Feature Profiles

veRoCE delineates diverse transport semantics designed for different applications. By implementing a simplified transport system focused solely on semantics for a specific scenario, the complexity and cost of hardware can be reduced. As a result, veRoCE introduces the notion of a “feature profile”, which is a mandatory set of transport functions that must be supported by an implementation. A “base” feature profile that only support RoCEv2 features must be supported by any veRoCE implementation, with the option to support additional profiles. A connection can only be initiated between two veRoCE endpoints that operate on the same feature profile, which is determined during the connection establishment phase. In its present specification, veRoCE outlines four feature profiles: Base (P0), P1, P2, and P3. Each profile has its own set of mandatory semantics, as detailed below table. # means the feature must be supported in a profile. - means the feature can be optionally supported in a profile.

Connection types	Features	Base (P0)	P1	P2	P3
RoCEv2 RC	RDMA Write/Write-imm	#	#	#	#
	RDMA Read	#	#	#	#
	Send/Send-imm	#	#	#	#
	Atomic	–	–	–	#
veRoCE RC	RDMA Write/Write-imm	–	#	#	#
	RDMA Read	–	–	–	#
	Send/Send-imm	–	–	–	#
	Atomic	–	–	–	#
	Selective Retransmission	–	–	#	#
	DDP	–	–	#	#
	Multi-path transmission	–	–	#	#
	Slow path detection	–	–	#	#
	Path-wise CC	–	–	#	#
	Packet trimming	–	–	#	#
	Shared Received Queue	–	–	–	#
UD (veRoCE and RoCEv2)	All Features	–	–	–	#

9 Communication Management

The veRoCE Connection Management (CM) protocol retains the RoCEv2 CM structure to ensure backward compatibility. The only key difference is that veRoCE adds QP-level protocol and feature profile negotiation to ensure endpoints use compatible configurations. Profile information is implicitly encoded in the Vendor ID field of CM packets (to avoid modifying the RoCEv2 CM packet format).

9.1 Vendor ID Encoding

The Vendor ID field (24 bits total) is split into two parts:

- Highest 20 bits: A unique vendor ID assigned to each veRoCE NIC.
- Low 4 bits: Feature profile identifier (only valid if the peer is a veRoCE NIC):
 - 0x0 : Base (p0)
 - 0x1 : p1
 - 0x2 : p2
 - 0x3 : p3
 - 0x4–0xF : Reserved (for future profiles).

9.2 Negotiation Process

The negotiation process ensures endpoints agree on a common protocol (veRoCE or RoCEv2) and feature profile:

1. A NIC supporting both veRoCE and RoCEv2 sends a REQ packet to UDP port 4791 (RoCEv2's port). The Vendor ID field includes the veRoCE vendor ID and the requested feature profile.
2. The responder replies with a REP packet to UDP port 4791, with two possible behaviors:
 - If the responder only supports RoCEv2 or the requested profile is unsupported: Sets Vendor ID to its own vendor ID + Base profile (falls back to RoCEv2).
 - If the responder supports the requested profile: Sets Vendor ID to its own vendor ID + requested profile.
3. The requester sends an RTU packet to UDP port 4791, with two possible modes:
 - veRoCE Mode: If the REP's Vendor ID and feature profile match the request.
 - RoCEv2 Mode: Otherwise.
4. After the RTU packet is sent, semantic packets and acknowledgment packets are transmitted to veRoCE's dedicated UDP destination port.

9.3 QPC Resource allocation

QP creation typically occurs before connection establishment, when both parties are unaware of which QP type to use. veRoCE recommends allocating QPC (Queue Pair Context) on both the host and NIC according to veRoCE specifications. During connection establishment, the QPC is cast to the corresponding type based on the protocol used.

10 Programmer's Guide to veRoCE

10.1 Operation Ordering Considerations

To minimize the effort required for developers to adopt veRoCE, most of its semantics align closely with RoCEv2. However, there are two key differences that must be addressed: First, due to the use of DDP, applications cannot assume any ordering of data written to the responder's memory by RDMA Writes or Sends. On the requester side, no ordering guarantee exists for data read via RDMA Reads. An application cannot determine the completion of a Send message by inspecting the last byte of the receive buffer. The only reliable indication of completion is the CQE of that message. Similarly, if an error CQE is posted for a message, the data from subsequent messages may have been written to the responder's memory.

Second, applications cannot assume the results of two RDMA Writes or Sends if they share the same target memory and the second message is posted before the first completes. This is because there is no ordering guarantee between Write and Send messages. If strict ordering is required, it is recommended to set the fence indicator on the second message to enforce operation ordering.

Aside from these two differences, applications can assume that veRoCE semantics are consistent with those of RoCEv2.

10.2 Enhanced Application Programming Interfaces

veRoCE introduces advanced features such as multi-path and lazy SACK, which require configuration parameters not covered by the standard verbs API. Examples of such parameters include the number of paths, the list of UDP source ports, and the lazy SACK threshold.

To ensure transparent adoption by existing applications utilizing the verbs interface, veRoCE advocates a host-based configuration approach. This approach preserves the existing verbs interface unchanged, thereby eliminating the need for any modifications within the application itself. Configuration APIs are provided via a separate, dedicated API library. These APIs are intended to be invoked by a management program, independent from the application processes. The configuration APIs are designed to support both global settings (applicable to all Queue Pairs) and granular, per-Queue Pair configurations.

It is important to note that veRoCE does not prescribe a specific API design or implementation. Should an application developer choose to modify the verbs interface, an implementation may optionally incorporate configuration channels directly into the verbs API.